Instructions

After completing the wiring shown in the “README” file, open Arduino IDE and paste into a blank sketch the code provided inside the folder named “Main Code”.

Main Code

The main program consists of a readily available example sketch from the official Adafruit library for the BNO055 board. This code is responsible for everything regarding the sensor itself, like initialization, data acquisition, and processing.

After extracting the roll, pitch, and yaw angles, we need a way to send the data to the PC. By treating the angles as separate axes, we can use a library named “Joystick”, which allows interfacing the sensor with a PC. At this point, it is important to note that this will only work with an **Arduino Leonardo** or **Micro**. This is because these two boards contain the **ATmega32u4** processor, enabling native USB communication with the PC without external circuitry. In simpler terms, these boards are recognized by a PC not just as a standard Arduino but rather as a mouse, keyboard, or, in our case, as an external joystick.

Using the “Joystick” library, we bind each sensor value (roll, pitch, yaw) to a virtual axis, which can then be recognized by our PC as if we had a physical joystick connected to it. At this point, we could feed these values straight into a game; however, without modifying them first, an obvious problem occurs.

If we want to look behind us, we would have to turn our heads around, but then we wouldn't be able to look at our screen. For that reason, a multiplier must be used such that, for example, 1° in real life is translated into 5° in-game. Having a translation ratio of, for example, 1-5 as described above, means that any movement of our head is 5 times larger inside the game, which could make it difficult if we wanted to focus on, let's say, instruments in front of us. Therefore, in addition to having a multiplier, we would also require that it is non-linear, meaning we would want a close to 1-1 ratio when looking ahead, and larger ratios the further back we want to look, allowing us to look behind us in-game while still being able to keep our eyes fixed on the screen.

Although we could bake this multiplier into the code, if we, for whatever reason, want to make a change, we would have to reprogram the board. It is much more intuitive and faster to use an external software. Fortunately, OpenTrack is just the software we want, and it's free.

After installing OpenTrack on the PC and opening it, we get a screen similar to the one shown below.

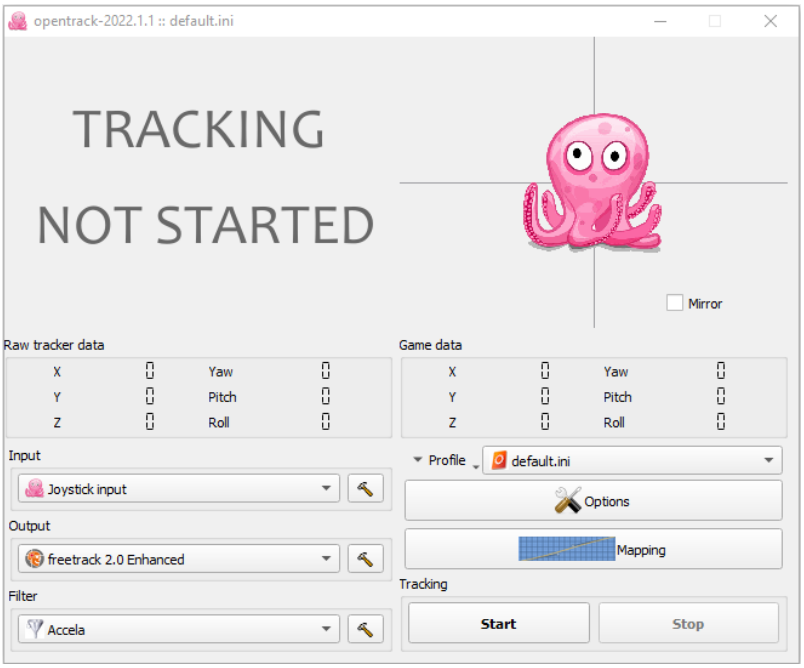

Fig 1. OpenTrack Environment

Starting at the bottom left, from the “Input” dropdown tab, select the “Joystick Input” option, for the “Output” choose “freetrack 2.0 Enhanced”, and for the filter choose “Accela” or the one that best fits your application. Then click the hammer icon ( ) for the “Input” menu, which should open a new window like the one below.

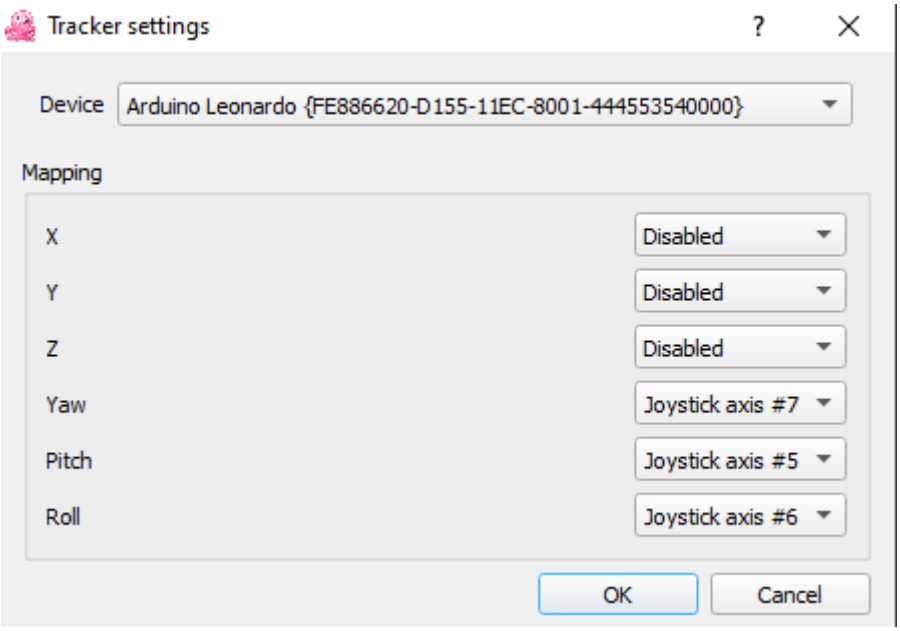

Fig 2. Tracker Settings Window

After making sure that the Arduino is connected to the PC and it is selected from the “Device” dropdown menu, as shown in the above window, we can assign the virtual joystick axis we created to the Yaw, Pitch, and Roll angles inside the OpenTrack software. Since in this project we only implement the roll, pitch, and yaw angles and not the physical movement of the head through space, we can ignore the first three menus named X, Y, and Z. By clicking on each of the bottom three dropdown menus, we can bind the axis inside the OpenTrack software. Keep in mind that the actual Joystick axis number may be different from the shown example. To check the correct binding, press “OK” after selecting the axes you want, and then press “Start” at the bottom center of the window. Press “Mapping”, and you should get a window as shown below.

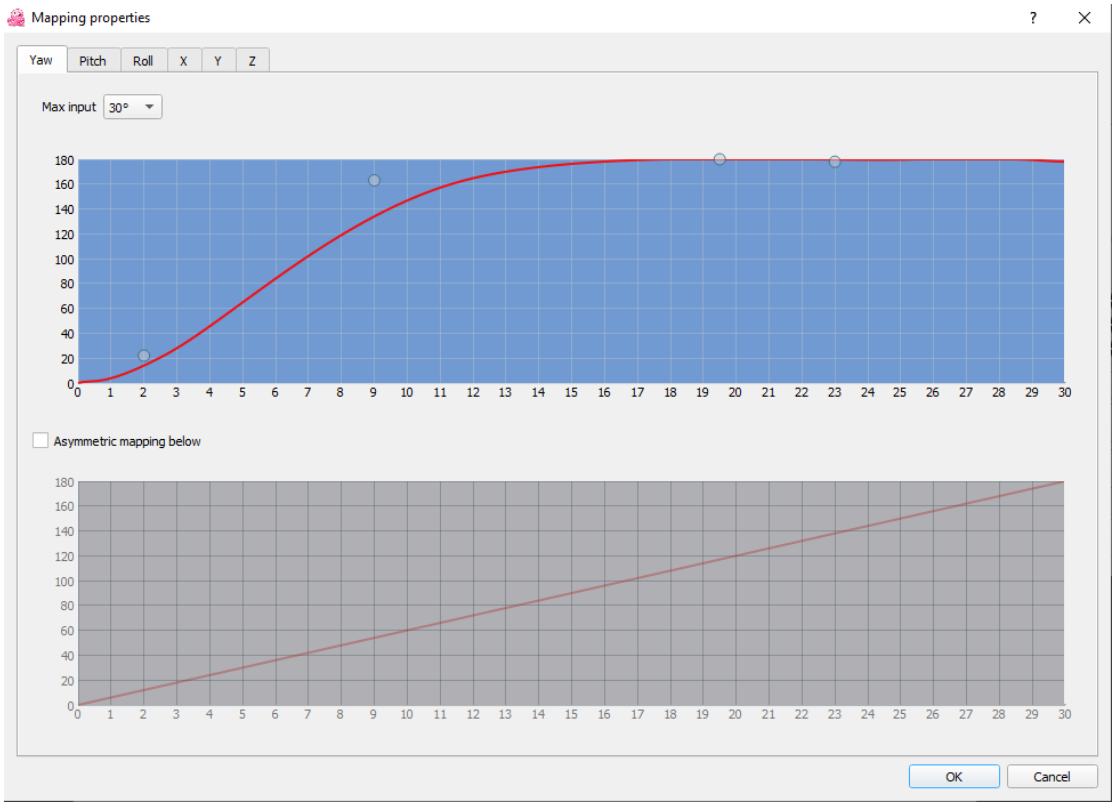

Fig 3. Mapping Properties Window

In the above graphs, the horizontal axis represents the input values from the sensor, and the vertical axis represents the software output that is going to be sent in the game. If the mapping is done correctly, by rotating the sensor along a particular axis, you should see a red dot moving along the shown path. If there's no movement, then go back to the "Tracker Settings" window (Fig 2) and bind a different Joystick axis. After being done correctly, you should be able to move the sensor along each axis and see the corresponding movement on the graphs in Fig 3. At this point, by clicking on top of the path, you can add nodes and shape the curve according to your needs. When satisfied with the behavior, you can press "OK".

Lastly, from the starting window (Fig 1) select "Options", opening the window shown in Fig 4.

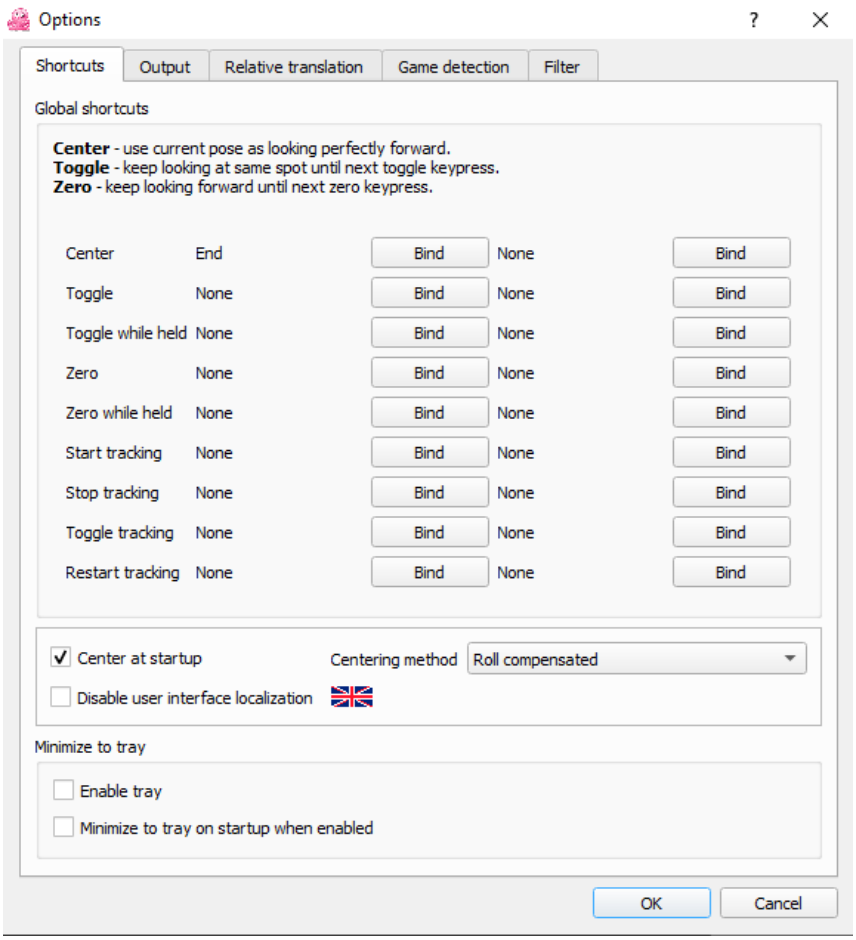

Fig 4. Options Window

From this window, go to the first tab from the top named “Shortcuts”. Here, you can assign shortcuts to perform certain actions within OpenTrack using the keyboard shortcuts. On the “Output” tab, you can select to invert a particular axis or modify it according to your needs. Lastly, on the “Filter” tab, you can modify the smoothing and dead zone of each axis.

Having completed these steps, you can log in to the game of your choice, and as long as it can recognize the OpenTrack software, you should be able to control the view of the in-game character through your own head movement.